

Library Management System

Project Report – Python Essentials (VITyarthi)

Name: Suyash Abhijit Sonar

Registration number: 26BAI10621

GitHub repository: https://github.com/SuyashSonar/Library_Managment_System

1. Introduction

Small libraries often track books in notebooks or spreadsheets. It gets messy when people forget who borrowed what or when it was due. For this project I built a command-line library management system in Python. It handles the everyday jobs: keeping a list of books, registering members, issuing and returning books, and working out fines for late returns.

2. Problem Statement

Build a program that lets a librarian manage books and members without paper records, and that handles due dates and late fines automatically. It must run completely from the terminal and keep its data between runs.

3. Objectives

- Store books and members and keep the data saved between sessions.
- Issue and return books while keeping the copy counts correct.
- Follow simple rules: a borrowing limit, and no issuing when no copies are left.
- Calculate fines automatically.
- Handle wrong input without crashing.

4. Features

- Add books (adding the same title and author again increases the copies).
- Show all books, search by title, author or id, and remove books.
- Register members; each gets an id like M001.
- Issue a book for 14 days, with a maximum of 3 books per member.
- Return a book and calculate the fine at Rs 2 per late day.
- Overdue report showing days late and the fine so far.
- Automatic saving to data/library.json after every change.

5. Requirements

- Python 3.8 or newer.
- Only the standard library is used (json, os, datetime). No pip install is needed.
- Any terminal (Command Prompt, PowerShell, Terminal). No GUI.

6. Design

The project is split into files by job, so each file is small and easy to understand:

- `main.py` – the menu, reading what the user types and printing results.
- `library.py` – a `Library` class that holds all the rules and does the saving and loading.
- `data/library.json` – created automatically; stores the data.

Data is stored in two dictionaries. "books" maps a book id to its title, author, total copies and available copies. "members" maps a member id to the name and a list of borrowed books, each with an issue date and a due date. Dictionaries make it easy to look things up by id. Three constants at the top of `library.py` control the rules: `LOAN_DAYS = 14`, `FINE_PER_DAY = 2` and `MAX_BOOKS = 3`.

7. How It Works

When the program starts, the `Library` object loads the JSON file if there is one. The menu then repeats until the user chooses 0. Each menu option calls one method of the `Library` class, and every method that changes data saves the file straight away. If the user does something invalid, the method raises a `LibraryError` and the menu prints a short message and shows the menu again. On return, the fine is $(\text{return date} - \text{due date}) \times 2$ when the result is positive, otherwise 0.

8. Functions Explained

8.1 `library.py`

Function	What it does
<code>LibraryError</code>	A custom exception. It is raised whenever the user does something wrong (bad id, no copies left, empty name) so the menu can show a friendly message instead of crashing.
<code>__init__(path)</code>	Creates the <code>Library</code> object, sets up the empty books and members dictionaries and calls <code>load()</code> .
<code>load()</code>	Reads <code>data/library.json</code> if it exists and fills the books and members dictionaries. If the file is missing it starts empty; if the file is damaged it prints a warning and starts empty.
<code>save()</code>	Writes both dictionaries to <code>data/library.json</code> . It creates the data folder if needed. Every function that changes something calls it, so nothing is lost.
<code>_next_id(prefix, existing)</code>	Makes the next id like B001 or M001 by taking the largest number already used and adding 1.
<code>_get_book(book_id)</code>	Cleans up the typed id (spaces, lowercase) and returns the book. Raises <code>LibraryError</code> if no such book exists.
<code>_get_member(member_id)</code>	Same as <code>_get_book</code> but for members.
<code>add_book(title, author, copies)</code>	Adds a new book and returns its id. If the same title and author (ignoring capital letters) already exist, it just adds to the number of copies instead of making a duplicate. Rejects empty text and copies below 1.
<code>remove_book(book_id)</code>	Deletes a book, but only if every copy is back on the shelf.
<code>search_books(query)</code>	Returns all books where the query appears in the title or author, or matches the book id. Not case sensitive.
<code>add_member(name)</code>	Registers a member with an empty borrowed list and returns the new member id. Rejects an empty name.

Function	What it does
issue_book(member_id, book_id)	Lends a book. It checks the member has fewer than 3 books, does not already hold this book, and that a copy is available. Then it records the issue date and due date (14 days later), reduces available copies by one and returns the due date.
return_book(member_id, book_id)	Takes a book back. It finds the record, works out the days late, fine = days late x Rs 2 (0 if on time), removes the record, adds the copy back and returns the fine.
overdue_list()	Goes through every member and returns a list of books whose due date has passed, with member name, book title and days late.

8.2 main.py

Function	What it does
main()	Creates the Library, shows the menu in a loop, runs the chosen action and catches LibraryError so the program never crashes on bad input. Option 0 exits.
ask_int(prompt, default)	Keeps asking until the user types a whole number. Pressing Enter uses the default when one is given.
print_books(books)	Prints books as a table with ID, title, author and available/total copies.
add_book(lib)	Menu option 1. Asks for title, author and copies and calls Library.add_book.
search_books(lib)	Menu option 3. Asks for a search word and prints matching books.
remove_book(lib)	Menu option 4. Asks for a book id and removes it.
add_member(lib)	Menu option 5. Asks for a name and registers the member.
show_members(lib)	Menu option 6. Lists every member with the books they currently have and the due dates.
issue(lib)	Menu option 7. Asks for member id and book id, issues the book and prints the due date.
give_back(lib)	Menu option 8. Asks for member id and book id, returns the book and prints the fine if there is one.
show_overdue(lib)	Menu option 9. Prints every overdue book with days late and the fine so far.

9. Testing

No.	Function tested	What I did	Expected	Result
1	add_book	Added "Wings of Fire" by A P J Abdul Kalam with 2 copies.	New book created with an id.	Pass. Got id B001.
2	add_book (duplicate)	Added the same book again in small letters with 1 copy.	No new entry; copies go up to 3.	Pass. Same id B001, list showed 3/3.
3	add_book (empty)	Left the title empty.	Error message, nothing added.	Pass. "Title and author can't be empty."
4	ask_int	Typed "abc" when asked for copies.	Asks again instead of crashing.	Pass. "Please type a number."
5	print_books	Chose "Show all books" after adding 4 books.	Table with all books and availability.	Pass. All 4 shown correctly.
6	search_books	Searched "kalam", then "zzz".	First finds the book, second finds nothing.	Pass. B001 shown; "Nothing to show."

No.	Function tested	What I did	Expected	Result
7	add_member	Registered "Asha" and "Ravi"; also tried an empty name.	Ids M001 and M002; error for empty name.	Pass.
8	issue_book	Issued B001 to M001.	Issued with a due date 14 days away.	Pass. Due 2026-10-13 (issued 2026-09-29).
9	issue_book (repeat)	Issued B001 to M001 again.	Refused, member already has it.	Pass. Error shown.
10	issue_book (bad ids)	Used member M999, then book B999.	Error for each.	Pass. "No member with id M999." / "No book with id B999."
11	issue_book (no copies)	Godan has 1 copy and Asha held it; tried to issue it to Ravi.	Refused, no copy available.	Pass. "No copies available right now."
12	issue_book (limit)	Asha borrowed 3 books, then tried a 4th.	Refused after 3 books.	Pass. "Asha already has 3 books."
13	remove_book (issued)	Tried to remove B001 while it was issued.	Refused.	Pass. "Some copies are still issued..."
14	remove_book	Removed B004 (not issued), then tried B999.	B004 removed; B999 gives an error.	Pass. B004 gone from the list.
15	return_book (on time)	Asha returned B001 on the same day.	Returned, no fine.	Pass. "Returned on time, no fine."
16	return_book (not borrowed)	Returned a book the member never took.	Error message.	Pass. "That member hasn't borrowed this book."
17	return_book (late)	Changed Ravi's due date in the JSON file to 5 days ago, then returned the book.	Fine of $5 \times 2 = \text{Rs } 10$.	Pass. "Fine to collect: Rs 10."
18	overdue_list	Opened the overdue report before and after the late return.	Shows the late book first; empty afterwards.	Pass. Showed 5 days late, Rs 10; then "No overdue books."
19	show_members	Chose "Show all members" after several issues.	Members with their books and due dates.	Pass.
20	save / load	Closed the program and started it again.	Books, members and loans still there.	Pass. Everything was restored.
21	main (menu)	Typed 77 at the menu.	Message and the menu again.	Pass. "That's not an option, try again."

Summary

All 21 test cases behaved as expected. Wrong input never crashed the program; it always showed a clear message and returned to the menu.

10. Challenges

- Testing fines: they depend on today's date, so I edited a due date in the JSON file to simulate a late return.
- Keeping copy counts right: issuing and returning must update both the book and the member, so both changes happen in the same function before saving.
- Bad input: typing letters where a number was expected crashed the program at first. I fixed it with the `ask_int` helper and the `LibraryError` approach.

11. Limitations and Future Scope

- Only one user at a time, because everything is in one JSON file. A database such as SQLite would suit a bigger library.
- No login for librarians and members.
- Fines are calculated but not stored as paid or unpaid.
- Possible additions: reservations, renewals, export to CSV, and a borrowing history for each member.

12. Conclusion

The project covers the core of what a small library needs and runs fully from the command line. Building it gave me practice with functions, dictionaries, file handling, exceptions and working with dates.